as well – in the database. If the passwords are stored in the database as plain text, an attack on the database will lose you all your user account passwords. A simple way to test if a website is storing passwords as plain text is to create an account, and use the forgot password feature. If the website is able to tell you your original password via email or on the site itself, they certainly have it stored somewhere or the other.

Nearly any website of note today uses some measure of session management and authentication, to provide users with a more dynamic environment. However, it is their responsibility to ensure that their website is secure from such attacks even if the website does not have much private information.

3.2.5 Insecure direct object references

"A direct object reference occurs when a developer exposes a reference to an internal implementation object, such as a file, directory, or database key. Without an access control check or other protection, attackers can manipulate these references to access unauthorised data."

Often in order to simply the working of a web site, the developers will directly expose many internal objects of the system. Such objects while not intended to be directly manipulated by the user could be altered by an attacker to their advantage.

To understand this more clearly, let us construct an imaginary messaging system, like that found in forums, or one like the direct messages feature of Twitter. In this application, each user has an email id, and can post messages to any email id on their friend list. Seems real enough?

Our hypothetical messaging system posts messages to users by sending them to a particular URL, which includes the user's email id and the message to post. For example:

http://www.vmessage.com/postmessage.php?from=someone@someplace.com&to=abcef@xyz.com&message=hey what's up

The developers of the postmessage.php script decided it would be simplest to just pass all the information in the URL itself and keep the script simple. Instead of checking whether the target user even belongs to the contact list of the logged in user, it assumes that the frontend would only be using this URL for emails in a person's contact list. For most users this might be the case, however for an attacker this is an excellent opportunity.

The application should validate the contact id in the database, and instead of using the email address directly, it should have used a more cryptic contacted which would only be valid for the current user. This could leave the URL as:

http://www.vmessage.com/postmessage.php? to=63&message=hey what's up

If the contact id here refers to a contact in the own users list, then the most